Koopman Operators Inside the Linux Scheduler: Burst Prediction, Closed-Loop Slice Control, Multi-Core Scheduling and Change Detection

Saarthak Manocha
Dept. of Information Technology
Chaitanya Bharathi Institute of Technology
Hyderabad, India
saarthakmanocha@gmail.com

***Abstract*—Koopman operator methods represent a nonlinear system by a linear operator on observables, estimated from data by (extended) dynamic mode decomposition (EDMD). We apply one such estimator, a least-squares operator on standardized observables, to the lowest layer of a computer: the Linux CPU scheduler. We record kernel traces on three machines (a 28-core bare-metal lab machine and two virtual machines, Intel and ARM) and check every reconstructed process against the kernel's own accounting: CPU time agrees within 0.93% (median) and run-queue wait within 0.22% over 105 processes. Validation exposed two measurement hazards: a tracing feedback loop of about 50,000 context switches per second, and a virtual-machine kernel that silently omits part of the switches out of idle. (i) *Burst prediction for shortest-job-first (SJF).* A log-scale EDMD forecast beats the textbook exponential average in 33 of 33 contended I/O-bound processes (median error 14% lower, Holm-corrected) and, on real programs (Redis, SQLite, Node.js, ffmpeg), in 32 of 36 threads (29% lower, $p = 2 \times 10^{-6}$); on a positive control with regular bursts the textbook estimator is better. Conformal intervals around the forecast hold 90% coverage, and one prediction costs 5.6 ns in C. (ii) *Closed loop.* Feeding predicted bursts to the stock EEVDF scheduler as per-task slice requests, with no kernel change, cuts the response time of short requests by 28–31% in every paired run on one and two CPUs (635,000 requests); slice sizing matters more than forecast accuracy. (iii) *Multi-core.* Prediction matters only while runnable CPU-bound work exceeds the cores, and preemption matters more than prediction quality. (iv) *Change detection.* A live, unprivileged Koopman detector reading `/proc/stat` detects 29 of 36 scripted load changes with a median delay of 0.18 s at 0.5–1.1% of one core, though a static distance score on the same state is as good. The same operator also forecasts network traffic (19–29% lower error) and, in the EIGENWORLD pipeline, marks the 2008 financial crisis. We release the capture kit, validated traces, the closed-loop controller and an interactive scheduler visualizer.**

***Index Terms*—Koopman operator, dynamic mode decomposition, CPU scheduling, EEVDF, shortest job first, kernel tracing, conformal prediction, change detection**

I. Introduction

Shortest-job-first (SJF) scheduling minimizes mean waiting time, but it needs the length of each process's next CPU burst before the burst runs. Operating systems textbooks therefore teach an estimate, the exponential average $\tau_{n+1} = \alpha t_n + (1-\alpha)\tau_n$ [1], and production schedulers sidestep prediction altogether: Linux's fair class, since kernel 6.6, picks tasks by earliest eligible virtual deadline first (EEVDF) [2], [3]. Whether a burst can be predicted better, and whether a better prediction would help a multi-core machine, are empirical questions that are rarely answered with real kernel data.

Koopman operator theory [4]–[6] offers a candidate. It represents a nonlinear system by a linear operator acting on functions of the state, so that a least-squares matrix fitted to data, as in dynamic mode decomposition (DMD) and its extension EDMD [7]–[10], becomes a forecaster whose eigenvalues describe how fast patterns decay. The author has used the same estimator before on aggregate network traffic [11] and in a market-crash prototype [12]. This paper asks how far it carries when the system is the CPU scheduler itself, and places those two higher-level applications alongside it.

Our emphasis is on measurement before modelling. Every scheduler number here comes from real kernel traces, and every reconstructed quantity is checked against counters the kernel keeps independently. We report results that favour the method and results that do not.

Contributions.

1) A validated measurement pipeline for scheduler dynamics: trace capture without perturbation, ground truth from `/proc/<pid>/schedstat`, and a self-test for missing events. Two hazards it caught are documented (Section IV).
2) Burst prediction for SJF on synthetic workloads, including a positive control, and on four real programs, with block-bootstrap intervals, multiple-testing correction, conformal prediction intervals and the cost per prediction (Section V).
3) A multi-core analysis combining measured run-queue latency on 1 to 28 cores with a trace-driven simulator of FCFS, round robin, SJF and SRTF (Section VI).
4) A closed-loop controller that steers the unmodified Linux EEVDF scheduler through per-task slice requests from predicted bursts, evaluated in 126 paired runs against stock Linux, hand-tuned settings and oracles (Section VII).

5) A machine-level Koopman change detector, evaluated offline on traces and live, as an unprivileged process, on 48 scripted runs on two architectures, against four baselines (Section VIII).
6) An interactive visualizer that replays real scheduler traces with a synchronized Koopman view (Section IX).
7) A summary of the same estimator at higher layers: network-traffic forecasting and a seeded reproduction of EIGENWORLD's market-crisis signal (Section X).

II. Background and Related Work

Koopman operators and DMD. For a map $x_{t+1} = F(x_t)$, the Koopman operator advances observables, $(\mathcal{K}g)(x) = g(F(x))$, and is linear even when F is not [4]. EDMD approximates it by the matrix $\mathbf{K}$ minimizing $\|\Psi_Y - \Psi_X \mathbf{K}\|_F$ over snapshot pairs of a chosen dictionary ψ [10]; with $\psi(x) = x$ this is DMD [7], [9], and EDMD converges to the projection of $\mathcal{K}$ onto the dictionary span [13]. Time-delay observables are justified by Takens' theorem [14], [15]. Measurement noise biases eigenvalues towards zero [16]. Applications include fluids, power grids and finance [17], [18].

Scheduling. SJF and its preemptive form SRTF are optimal for mean waiting time on one processor given exact lengths; size-based scheduling helps heavy-tailed workloads in practice [19]. Linux replaced the CFS pick logic with EEVDF in 6.6 [3]; sched_ext (6.12) allows custom policies [20]. Scheduler behaviour is usually observed with `perf sched` [21] or ftrace [22]; per-task and per-CPU counters are exposed in `schedstat` [23].

Early warning. Rising autocorrelation and slowing recovery before critical transitions [24] motivate eigenvalue-based precursors; diffusion maps [25] and NLSA [26] provide nonlinear coordinates on which such spectra can be computed, the design used in EIGENWORLD [12].

III. One Estimator for Three Layers

Throughout, a state $s_t \in \mathbb{R}^p$ (a vector of measured quantities) is standardized with training statistics, $z_t = (s_t - \mu)/\sigma$, and one operator is fitted by least squares,

$$\mathbf{K} = \arg\min_A \sum_t \|z_{t+1} - z_t A\|^2 = Z_X^+ Z_Y. \tag{1}$$

For burst prediction the state is a delay embedding of the last three bursts, optionally lifted with squares and products (*poly*) or taken on a log scale (*log*). For change detection the state describes a whole machine, and the score is the one-step residual

$$r_t = \frac{\|z_{t+1} - z_t \mathbf{K}\| - \bar{r}}{s_r}, \tag{2}$$

in units of the training residual mean $\bar{r}$ and spread s_r, averaged over a short trailing window; an alarm is raised when it stays above a threshold for a hold time. All smoothing is causal. The same code is used in every domain.

TABLE I
Platforms and Data

Platform	Kernel	Capture	Runs
Lab, HP Pro Tower, 28 CPUs, bare metal	7.0	perf	1 (75 s)
Intel Xeon VM, 2 vCPUs (KVM)	6.18	tracefs	12 + 24 + 3
same VM, closed loop	6.18	counters	126
Apple M-series host, ARM VM, 4 vCPUs	6.8	counters	9 + 24

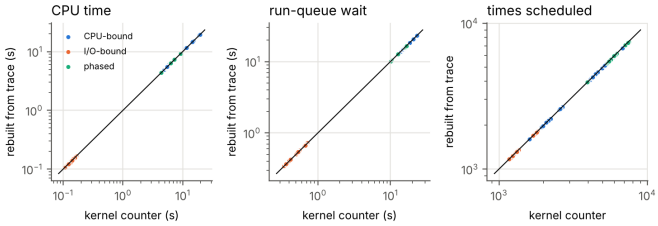


Fig. 1. Every reconstructed process against the kernel's own counters (105 processes, 12 captures).

IV. Measurement and Validation

Platforms and workloads (Table I). All kernels are 6.6 or later, so all run EEVDF. Workloads are small Python programs: a CPU-bound loop; an I/O-bound loop that writes 10 kB and sleeps 50 ms; and a *phased* positive control that alternates every 2 s between bursts of 1 ms and 20 ms of CPU time, so its bursts have known structure. The lab trace (*Trace 3*) ran three CPU-bound and three I/O-bound jobs; the virtual-machine (VM) captures ran these plus two phased jobs, pinned to 1 or 2 cores, overloaded, or with a scheduled load drop, each repeated three times. Real programs are covered in Section V and the closed-loop workload in Section VII.

Reconstruction. From `sched_switch`, `sched_wakeup` and `sched_migrate_task` events we rebuild, per process, every interval on a CPU (with the state at switch-out: preempted, sleeping or disk wait), every interval waiting in a run queue (after a wake-up or a preemption), and every migration.

Ground truth. Each workload saves, just before it exits, the kernel's own `/proc/self/schedstat` (CPU time, run-queue wait, times scheduled) and context-switch counts. These are maintained independently of tracing. Over 105 processes in 12 VM captures (Fig. 1), reconstructed CPU time differs by 0.93% (median; max 3.45%), wait time by 0.22% (max 2.07%), and the number of times scheduled by a median of 4 out of thousands (max 0.57%), with no lost events. The excess CPU time in the trace comes from the exit path after the counters are read and from interrupt time, which the kernel's CPU accounting excludes.

Hazard 1: the observer effect. A first capture tool streamed events out of the kernel while recording. Each event woke the reader, which preempted the workload and produced further events: a loop of about 50,000 context switches per second on any core with spare capacity. Traces grew from 0.5–0.8 MB to 15–23 MB and counts disagreed with the kernel by up to 9%. Recording into a large kernel buffer and reading it once

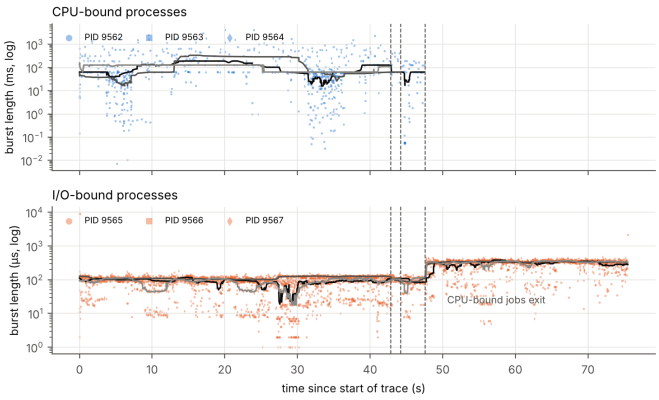


Fig. 2. Trace 3 (bare metal, 28 cores): every burst over time. I/O bursts lengthen about threefold on the same cores when the CPU-bound jobs exit (dashed lines).

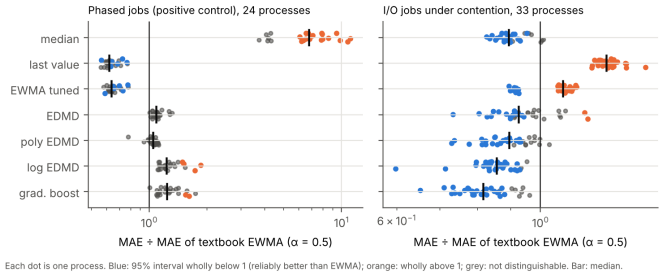


Fig. 3. Error relative to the textbook exponential average, one dot per process. Blue/orange: 95% interval wholly below/above 1.

after the run removed it. One contaminated capture is kept as evidence.

Hazard 2: missing idle exits on a VM. On the Intel VM's kernel, part of the `sched_switch` events for a CPU leaving idle are never emitted; tracefs and perf agree, and the ring buffer reports no loss. A task that slept 201 times was seen switching back in only 1 to 176 times across seven self-tests. The lab machine does not show it (zero unmatched switch-outs in Trace 3). We did not find this documented and did not determine its cause. Our parser counts unmatched switch-outs, the capture tool runs a self-test, and the VM experiments keep every core busy; with busy cores the counts match the kernel (on one core, within 4 switches for every process).

V. Burst Prediction for SJF

Methods. Every method predicts a burst from earlier bursts of the same process only: training mean and median, last value, the textbook exponential average with $\alpha = 0.5$ (EWMA) and with α tuned on training data, EDMD one-step forecasts (linear, poly, log; delay 3), recursive EDMD, and gradient boosting on the same three lags. The last 20% of each process is the test set; intervals are 95% moving-block bootstrap intervals [27].

Uncontended bursts (Trace 3). On the idle 28-core lab machine (on average 2.3 cores busy), CPU-bound bursts end only because kernel threads preempt them: all 769 preemptions came from `kworker` (55%) or `migration` threads (45%). I/O bursts roughly tripled, from 85–113 μs to 320–340 μs, on the *same* cores when the CPU-bound jobs exited (Fig. 2); we attribute this to power management, which we could not measure. That level shift dominates I/O forecasting: methods that follow recent bursts cut error by 52–83% against a constant, and the textbook EWMA was best or tied. Recursive EDMD forecasts decay to the mean, as the operator's eigenvalues (all inside the unit circle) imply. The I/O operators' dominant eigenvalue, 0.88–0.97, is the Koopman signature of the level shift.

Bursts under contention (VM). Here a burst is the CPU time between two blocking events. Results over 57 processes are in Fig. 3. On the positive control (24 phased processes), every method that uses recent bursts cuts error by 90–96% against the mean, so the methods find structure when it exists; the best is tuned EWMA (better than $\alpha = 0.5$ in 24/24 processes) and the EDMD variants are 5–23% *worse* than the textbook estimator (median ratios 1.05–1.23). On I/O bursts under contention (33 processes) the ordering reverses: log-scale EDMD beats the textbook EWMA in 33/33 processes (median error ratio 0.86), polynomial EDMD in 32/33 and gradient boosting in 33/33. With one sign test per method and Holm correction over all ten comparisons [28], these are significant ($p_{\text{Holm}} = 2 \times 10^{-9}$ for log EDMD and gradient boosting, 6×10^{-8} for poly EDMD). Contended I/O bursts are noisy and the exponential average overreacts to single long bursts; the log-scale operator does not.

Real programs. To test beyond synthetic loops we traced, at the same time on both VM cores, a Redis 7.0 server driven by a paced client (GET, SET and occasional range and sort commands), a Node.js 22 HTTP server with alternating phases of light and heavy JSON requests, SQL queries of mixed size against SQLite 3.45, real-time H.264 encoding with ffmpeg (all encoder threads), and the two load generators, with a parallel gcc build and xz compression as untracked background load; three captures of 60 s. One lowest-priority (`SCHED_IDLE`) spinner per core keeps the cores out of the idle path that loses events on this VM. Over the 36 threads with at least 60 bursts, CPU time agrees with `schedstat` within 1.8% and wait time within 1.5% (medians); 28 threads are within 5%, and the others are ffmpeg helper threads that run about 12 μs per slice. With the same methods and split, log-scale EDMD beats the textbook EWMA in 32 of 36 threads with a median error ratio of 0.71 (sign test $p = 2 \times 10^{-6}$), polynomial EDMD in 35 of 36 (0.81), gradient boosting in 34 of 36 (0.79) and tuned EWMA in 29 of 36 (0.94) (Fig. 4). The gain is largest for Redis and SQLite (median ratios 0.55 and 0.56), whose request bursts are short with occasional long ones, which an exponential average chases and a log-scale operator does not.

Spectra. Fitted on windows of 150 bursts, the operator's three eigenvalues alone identify which of the five pro-

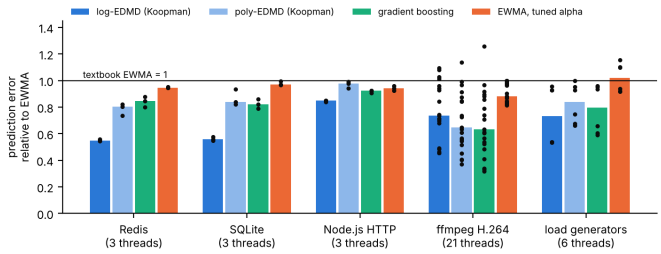


Fig. 4. Real programs: burst-prediction error relative to the textbook exponential average (bars: median; dots: threads; three captures).

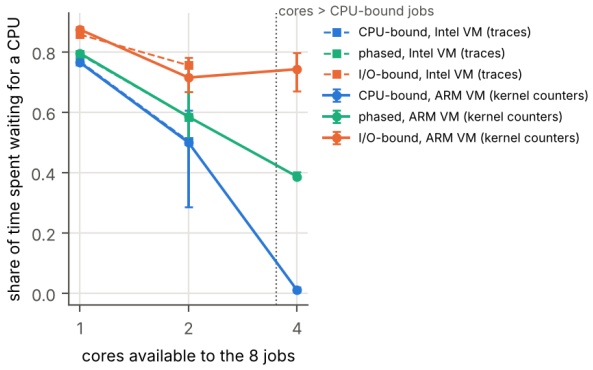


Fig. 5. Share of time spent waiting for a CPU, same 8 jobs on 1, 2 and 4 cores, two architectures, three repetitions.

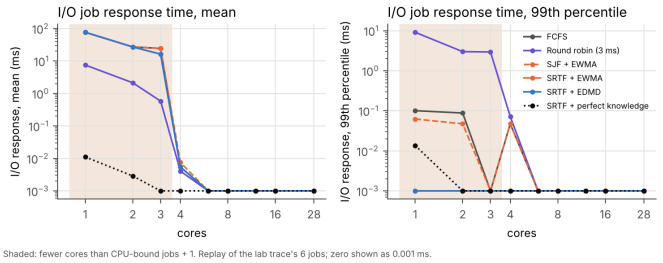


Fig. 6. Simulated I/O response time versus core count, replaying the lab trace's six jobs.

grams a window came from with 76% balanced accuracy (chance 20%; leave-one-capture-out), with readable signatures: SQLite's eigenvalues are all real, the load generators have the slowest mode ($|\lambda_1| = 0.87$), Redis and Node.js the fastest decay ($|\lambda_1| \approx 0.45$). Simple burst statistics classify better (99.8%), so we read the spectrum as a description, not a classifier.

Uncertainty and cost. A scheduler acting on a forecast needs to know how wrong it may be. Split conformal prediction [29], [30] on log-errors (fit on 60%, calibrate on 20%, test on 20% of each of 60 processes) gave the target 90% coverage (median 0.90 for log-EDMD, 0.91 for EWMA on I/O processes), with the Koopman interval narrower in 34 of 60 processes (median width ratio 0.84). In C, a three-lag operator predicts the next burst in 5.6 ns and updates itself by recursive least squares in 29 ns, less than the cost of one context switch; the cost of a Koopman predictor is not an obstacle to running it inside a scheduler.

VI. Multi-Core Scheduling

Measured latency. On the lab machine, processes waited a median 16 μs for a CPU; no queue formed, so the order a scheduler picks in could not matter. On one VM core the same kind of workload waited a median 0.07 ms (I/O) and 2.6 ms (phased) after waking, with 95th percentiles of 2.6 and 7.9 ms. On the ARM VM, using only the kernel's per-process counters, CPU-bound jobs spent 76% of their time waiting on one core, 50% on two, and 1% on four (Fig. 5); the Intel VM traces give the same values on one and two cores.

Trace-driven simulation. Each traced process is replayed as a closed loop of its real CPU bursts and real blocking times on m identical cores with one shared queue, under FCFS, round robin (3 ms), non-preemptive SJF and preemptive SRTF, each with a predictor (running mean, EWMA, online log-EDMD, or exact lengths). A job that has run past its prediction is assumed to need at least as long again. Replaying the lab's six jobs (Fig. 6), I/O response time is zero for every policy from four cores up, the number of CPU-bound jobs plus one. Below that, non-preemptive SJF behaves like FCFS whatever the predictor (two cores: 10.2 versus 10.4 ms mean I/O response), because an I/O job that arrives while a CPU-bound job holds the core must wait for it; SRTF with any predictor brings it to 0.011 ms. Measured EEVDF on one core gave interactive jobs a 1.87 ms mean response, against 1.5–1.9 ms for simulated SRTF with predictors, 6.8 ms for round robin and 16.4 ms for FCFS. The simulator omits context-switch and migration costs and per-core queues.

VII. Closing the Loop: Steering EEVDF

Since kernel 6.12, EEVDF lets a task request its own time slice through `sched_setattr` [31]. A shorter slice gives an earlier virtual deadline, so the task is served sooner after it wakes. Setting each task's slice to its predicted next burst therefore expresses SJF inside the stock scheduler, with no kernel patch and no new scheduling class.

Controller. A user-space agent polls `/proc/<pid>/schedstat` and `status` every 10 ms, extracts each task's bursts (CPU time between voluntary switches), predicts the next one and sets the slice to the prediction, clamped between the kernel minimum (0.1 ms) and the default slice (1.4 ms); it only ever shortens slices. Predictors are the textbook EWMA and log-EDMD (three lags, refitted every 20 bursts on the last 300); each also runs with a conformal margin, the prediction multiplied by the 90th percentile of the task's recent error ratios. References are stock Linux, a 0.1 ms slice for every task, a hand-labelled 0.1 ms slice for request tasks only, and two oracles in which each task sets its slice to its true next burst, or to twice it.

Workload. Request tasks (C) wake on a period of 15–25 ms and do one CPU burst each, drawn from a two-regime log-

autoregressive process (short bursts near 0.3 ms, long ones near 4 ms, switching with probability 0.05 per request), and log deadline, wake-up and finish times; CPU hogs spin and count the CPU time they receive. One CPU ran three hogs and four request tasks (eight repetitions of 20 s), two CPUs six and eight (six repetitions). Each repetition runs every policy with the same seeds in shuffled order (126 runs, 635,531 requests). The request tasks' own CPU accounting agrees with `schedstat` within 0.8% (median).

Results (Fig. 7). Both predictors cut the mean response time of short requests (under 1 ms of CPU) in every paired repetition: Koopman by 31% on one CPU (1.48 against 2.14 ms) and 28% on two, EWMA by 29% on both (Wilcoxon $p = 0.008$ and 0.03, the smallest attainable with 8 and 6 pairs). Koopman was 2.6% better than EWMA on one CPU (7 of 8 pairs, $p = 0.016$) and level on two. The oracles show what drives the gain: a slice exactly as long as the burst is no better than stock Linux (+3%), because the burst slightly overruns its slice and the task falls behind the hogs, while twice the burst gives -46%. Hand-labelled and uniformly short slices cut wake-up latency but chop every burst (7–10 times more context switches) and do not shorten responses. The conformal margin helps EWMA (to -35%) but not Koopman (-20%), whose wider log-scale intervals push slices towards the default. The price is SJF's usual one: long requests take 4–7% longer and mean response over all requests changes by $+0.5$ to $+4\%$. CPU hogs lose 3–7% of their CPU time, which matches the agent's own CPU time almost exactly (for example 0.57 s lost against 0.60 s used by the Koopman agent in 20 s); this is the cost of a Python agent polling `/proc`, not of the scheduling decision.

VIII. Koopman View of the Whole Machine

Offline, from traces. The machine state every 10 ms is the busy fraction and queued tasks of each core plus the CPU share of each job class. In three VM runs where four of six CPU-bound jobs stop at 20 s while every core stays fully busy, an operator fitted on the first 8 s raised an alarm 77–94 ms after the change in all three runs, with no false alarm before it or in nine runs without a change (Fig. 8). A persistence residual and a distance-from-normal score on the same inputs detected none of the three; a utilization monitor cannot, because utilization stays at 100%. After the change, the residual also shows the phased jobs' 2 s cycle. On the 28-core lab trace, a 52-variable per-core state gave false alarms for every detector; a four-variable machine-wide state, added after seeing this, detected the first CPU-bound exit 32 ms after it without a false alarm (distance-from-normal: 52 ms).

Live, from counters. For a deployable version, `live_monitor.py` samples `/proc/stat` every 10 ms and forms a 100 ms state of busy cores, runnable tasks and context switches per second; it needs no root and no tracing, and fits its operator on the first 8 s. (`/proc/schedstat` per-CPU run time was rejected because it is credited only when a task leaves a core.) We ran 24 scripted runs on each VM: at a random time between 15 and 25 s, one CPU-bound job or one per core was added or stopped, or two or $2N$ fast I/O jobs started; six runs per platform had no change. The monitor ran as a separate process and was not told when or whether a change happened. Online, it detected 17/18 changes on the ARM VM (median delay 0.12 s) and 12/18 on the Intel VM (0.26 s) and used 0.5% and 1.1% of one core; it raised at least one false alarm in 10 of 24 runs on each platform.

Against baselines (Fig. 9). Utilization and run-queue monitors raised few false alarms (0–4 runs) but missed every I/O burst on the ARM VM, which shows up only in the switch rate. A distance-from-normal score on the same three variables detected 18/18 and 16/18 changes. Across 54 settings of training length, smoothing and threshold, the Koopman residual was never strictly better than that static score and was strictly worse in 51. On this state, watching the variables jointly matters; modelling their dynamics adds nothing.

IX. Making the Scheduler Visible

The Scheduler Observatory (Fig. 10) replays the validated traces in slow motion in a web browser. Each core is a tile lit in the colour of the process it runs; queued processes sit under their core with a bar that grows while they wait; sleeping and disk-blocked processes rest in a pool; migrations are drawn as arcs. A log narrates each scheduling decision with its cause and the wait that preceded it, and a Koopman panel shows, in sync, the eigenvalues of a sliding-window operator on the unit circle and the live residual score. All values come from the reconstructed timelines above.

X. Higher-Layer Applications

The estimator used for the scheduler, a least-squares operator on a standardized state, has also been applied above the operating system. We report these as case studies of the same method rather than as a general evaluation at those layers.

Network traffic. In earlier work [11], 16.2 million CSE-CIC-IDS2018 flow records [32] were binned into 9,777 windows of 30 s, each summarized by an 11-dimensional state: ports targeted, protocol diversity, flow count, forward and backward volume, their asymmetry, mean and spread of inter-arrival time, and SYN, RST and FIN counts. One 11×11 operator was fitted on 6,871 training transitions that do not cross capture gaps. On the chronologically held-out final day (1,046 transitions) it lowered mean squared forecast error against persistence by 25.9% at 30 s, 29.1% at 60 s and 18.9% at 15 min, and the same construction lowered it by 20.7% on UNSW-NB15 [33] (27,212 forecasts). All eleven eigenvalues are real and inside the unit circle; the largest, 0.995, gives a slowest-mode half-life of 71 min, so aggregate traffic carries slow, persistent structure that a linear operator captures.

Equity markets. EIGENWORLD [12] builds diffusion-map coordinates [25] from seven daily S&P 500 features (return, 21- and 63-day volatility and return, their ratio, volume z-score), fits a Koopman operator on them and tracks the spectral entropy of the embedding. Its kernel bandwidth is drawn from an unseeded random sample, so we reran its unmodified code

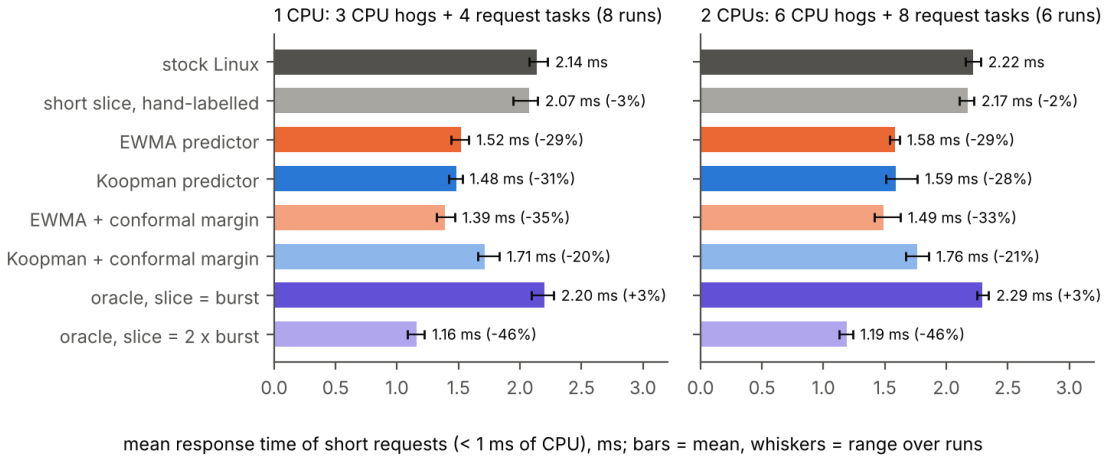


Fig. 7. Closed loop on the stock EEVDF scheduler: mean response time of short requests per slice policy (bars: mean over paired repetitions; whiskers: range).

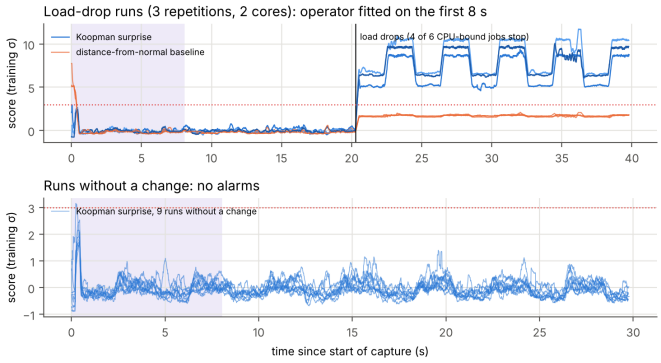


Fig. 8. Trace-based detector on the VM. Top: three runs with a load drop at 20 s. Bottom: nine runs without a change. Shaded: training.

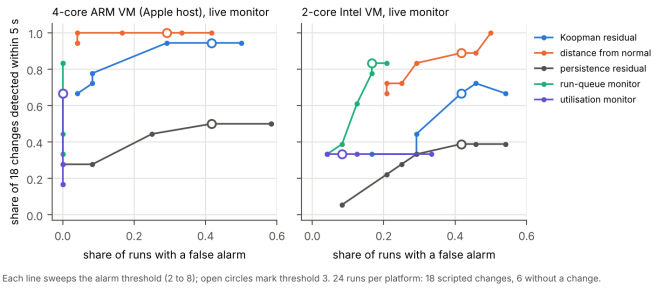


Fig. 9. Live change detection, 24 runs per platform. Each line sweeps the threshold; circles mark threshold 3.

under 11 fixed seeds. On the original 2004–2010 data, 10 of 11 seeds reproduce the published result: the minimum spectral entropy falls on 2008-08-27, 19 days before the Lehman Brothers bankruptcy (values 1.960–1.982; published 1.972). Extended to 1992–2026, the lowest entropy in 34 years still falls in July–August 2008 for all 11 seeds (Fig. 11).

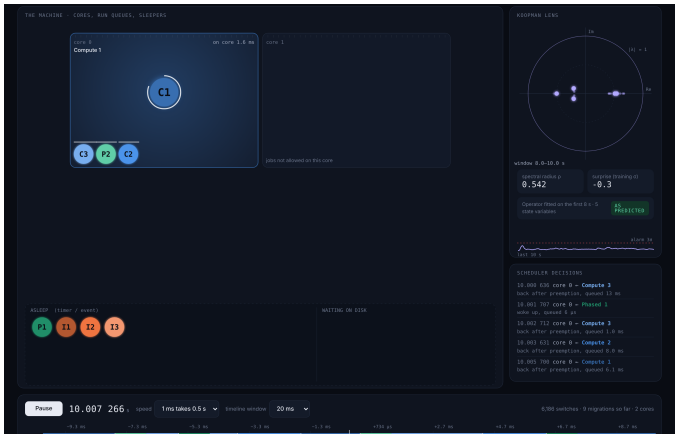


Fig. 10. Scheduler Observatory replaying one core shared by eight jobs: the running job, three queued jobs with wait bars, sleepers, the Koopman eigenvalues and the decision log.

XI. Discussion

Table II collects the results. One least-squares operator helped where the task was forecasting a noisy signal with short memory: contended I/O bursts, real server and database workloads, and aggregate network traffic. Forecasts are also useful in practice: fed to the unmodified scheduler as slice requests, they made short requests about 30% faster, at a predictable cost to long ones. In that loop, how the forecast is turned into a slice mattered more than the forecaster. It was a useful diagnostic: eigenvalues near one flagged the I/O level shift in Trace 3, and the residual exposed the phased jobs' cycle. It did not beat simpler tools where the task was noticing that something changed: on low-dimensional scheduler counters a static distance score was as good. The positive control shows the burst predictors are sound; where they lose, the structure is not there, or simpler estimators capture it.

For SJF on multi-core machines, the decisive quantity is

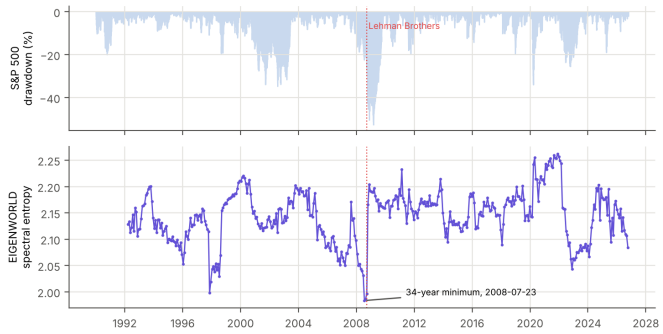


Fig. 11. S&P 500 drawdown and EIGENWORLD's spectral entropy (seed 0), 1992–2026. The 34-year minimum falls in July 2008, before the Lehman Brothers bankruptcy (dotted line).

TABLE II
WHERE ONE LINEAR OPERATOR HELPED

Task	Result	Compared with
Contended I/O bursts	14% lower error, 33/33 processes	textbook EWMA
Real programs	29% lower error, 32/36 threads	textbook EWMA
Steering EEVDF	short requests 28–31% faster, every run	stock Linux
Structured bursts	5–23% higher error	textbook EWMA
Load change, traces	3/3, 77–94 ms, no false alarm	baselines 0/3
Load change, live	29/36, 0.18 s, 20/48 runs false alarm	static score as good or better
Network forecast	18.9–29.1% lower MSE	persistence
Market crisis (2008)	minimum entropy 19 days before Lehman	10/11 seeds

runnable CPU-bound work relative to cores, not prediction accuracy. With spare cores nothing waits; with too few, preemption decides. EEVDF's preemption is already close to predicted SRTF for interactive jobs, and slice requests are the lever through which a predictor can improve on it.

Threats to validity. Two of three platforms are virtual machines; hypervisor scheduling adds noise, and one VM kernel drops idle exits, which we designed around. Most workloads are synthetic; the real-program captures cover four programs on one VM, and the closed-loop request tasks are synthetic. Only one bare-metal trace exists, with no contention. The simulator is idealized. The closed-loop agent polls every 10 ms, so it sees bursts later than an in-kernel predictor would. The trace-based regime results rest on three changes; the live results on 36. The machine-wide lab state follows earlier work; the four-variable lab state was chosen after seeing the per-core result and is reported as such. Network numbers are taken from the original study's outputs; the market result is a reproduction of one published event.

XII. Conclusion and Future Work

Measured carefully, the Linux scheduler is predictable where it is contended, and a Koopman operator is a good forecaster there: it beats the textbook SJF estimator on noisy contended I/O bursts and, by a wider margin, on real programs. Its forecasts, given to the unmodified EEVDF scheduler as slice requests, make short requests about 30% faster, and a prediction costs nanoseconds. For deciding whether something changed, the same operator is not better than simpler statistics on the same inputs. All of these results come from one validated pipeline, which we release with the data, the controller and the visualizer.

Remaining work needs the bare-metal lab machine: contended captures on 8 and 16 cores, the closed-loop and real-program experiments on bare metal, and a capture with CPU frequency recorded and the performance governor fixed, to test the power-management explanation of the threefold I/O slowdown. Further directions are moving the predictor and slice control into the kernel or a sched_ext policy [20], sizing slices from the forecast distribution rather than its centre, and running the detector protocol on window-level network data.

Data and Code Availability

Capture kit, live monitor, closed-loop controller, parsers, validation, simulator, all analyses, the 72 validated runs, the 126 closed-loop runs, the EIGENWORLD reproduction, and the visualizer are at https://github.com/SaarthakManocha/Koopman-Operator-Theory-Research, whose README lists the script that regenerates each table and figure.

References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Wiley, 2018.
[2] I. Stoica and H. Abdel-Wahab, "Earliest eligible virtual deadline first: A flexible and accurate mechanism for proportional share resource allocation," Old Dominion University, Tech. Rep. TR-95-22, 1995.
[3] J. Corbet, "An EEVDF CPU scheduler for Linux," LWN.net, Mar. 2023, https://lwn.net/Articles/925371/.
[4] B. O. Koopman, "Hamiltonian systems and transformation in Hilbert space," *Proc. Natl. Acad. Sci. USA*, vol. 17, no. 5, pp. 315–318, 1931.
[5] I. Mezić, "Spectral properties of dynamical systems, model reduction and decompositions," *Nonlinear Dyn.*, vol. 41, no. 1–3, pp. 309–325, 2005.
[6] S. L. Brunton, M. Budišić, E. Kaiser, and J. N. Kutz, "Modern Koopman theory for dynamical systems," *SIAM Rev.*, vol. 64, no. 2, pp. 229–340, 2022.
[7] P. J. Schmid, "Dynamic mode decomposition of numerical and experimental data," *J. Fluid Mech.*, vol. 656, pp. 5–28, 2010.
[8] C. W. Rowley, I. Mezić, S. Bagheri, P. Schlatter, and D. S. Henningson, "Spectral analysis of nonlinear flows," *J. Fluid Mech.*, vol. 641, pp. 115–127, 2009.
[9] J. H. Tu, C. W. Rowley, D. M. Luchtenburg, S. L. Brunton, and J. N. Kutz, "On dynamic mode decomposition: Theory and applications," *J. Comput. Dyn.*, vol. 1, no. 2, pp. 391–421, 2014.
[10] M. O. Williams, I. G. Kevrekidis, and C. W. Rowley, "A data-driven approximation of the Koopman operator: Extending dynamic mode decomposition," *J. Nonlinear Sci.*, vol. 25, no. 6, pp. 1307–1346, 2015.
[11] S. Manocha, R. R. Lingala, C. S. M. Sadhana, S. Kedia, C. S. M. Sahithi, and P. Vemuri, "Koopman operator theory for network traffic dynamics: An inspectable linear world model for attack forecasting," 2026, manuscript, Chaitanya Bharathi Institute of Technology.
[12] S. Manocha, "EIGENWORLD: Causal-topological intelligence for complex systems," Source code, https://github.com/SaarthakManocha/eigenworld, 2026.
[13] M. Korda and I. Mezić, "On convergence of extended dynamic mode decomposition to the Koopman operator," *J. Nonlinear Sci.*, vol. 28, no. 2, pp. 687–710, 2018.
[14] F. Takens, "Detecting strange attractors in turbulence," in *Dynamical Systems and Turbulence, Warwick 1980*, ser. Lecture Notes in Mathematics. Springer, 1981, vol. 898, pp. 366–381.

[15] H. Arbabi and I. Mezić, "Ergodic theory, dynamic mode decomposition, and computation of spectral properties of the Koopman operator," *SIAM J. Appl. Dyn. Syst.*, vol. 16, no. 4, pp. 2096–2126, 2017.
[16] S. T. M. Dawson, M. S. Hemati, M. O. Williams, and C. W. Rowley, "Characterizing and correcting for the effect of sensor noise in the dynamic mode decomposition," *Exp. Fluids*, vol. 57, no. 3, p. 42, 2016.
[17] J. N. Kutz, S. L. Brunton, B. W. Brunton, and J. L. Proctor, *Dynamic Mode Decomposition: Data-Driven Modeling of Complex Systems*. Philadelphia, PA: SIAM, 2016.
[18] J. Mann and J. N. Kutz, "Dynamic mode decomposition for financial trading strategies," *Quant. Finance*, vol. 16, no. 11, pp. 1643–1655, 2016.
[19] M. Harchol-Balter, B. Schroeder, N. Bansal, and M. Agrawal, "Size-based scheduling to improve web performance," *ACM Trans. Comput. Syst.*, vol. 21, no. 2, pp. 207–233, 2003.
[20] The Linux kernel documentation, "Extensible scheduler class (sched_ext)," https://docs.kernel.org/scheduler/sched-ext.html, 2024.
[21] Linux perf, "perf-sched(1): Tool to trace/measure scheduler properties (latencies)," Linux manual page.
[22] The Linux kernel documentation, "ftrace – function tracer," https://docs.kernel.org/trace/ftrace.html.
[23] ——, "Scheduler statistics," https://docs.kernel.org/scheduler/sched-stats.html.
[24] M. Scheffer, J. Bascompte, W. A. Brock, V. Brovkin, S. R. Carpenter, V. Dakos, H. Held, E. H. van Nes, M. Rietkerk, and G. Sugihara, "Early-warning signals for critical transitions," *Nature*, vol. 461, pp. 53–59, 2009.
[25] R. R. Coifman and S. Lafon, "Diffusion maps," *Appl. Comput. Harmon. Anal.*, vol. 21, no. 1, pp. 5–30, 2006.
[26] D. Giannakis and A. J. Majda, "Nonlinear Laplacian spectral analysis for time series with intermittency and low-frequency variability," *Proc. Natl. Acad. Sci. USA*, vol. 109, no. 7, pp. 2222–2227, 2012.
[27] H. R. Künsch, "The jackknife and the bootstrap for general stationary observations," *Ann. Statist.*, vol. 17, no. 3, pp. 1217–1241, 1989.
[28] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.
[29] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. New York: Springer, 2005.
[30] A. N. Angelopoulos and S. Bates, "Conformal prediction: A gentle introduction," *Found. Trends Mach. Learn.*, vol. 16, no. 4, pp. 494–591, 2023.
[31] The Linux kernel documentation, "EEVDF scheduler," https://docs.kernel.org/scheduler/sched-eevdf.html, 2024.
[32] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Security Privacy (ICISSP)*, 2018, pp. 108–116.
[33] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in *Proc. Military Commun. Inf. Syst. Conf. (MilCIS)*, 2015, pp. 1–6.